Практическая работа

Тема: Lead Architecture Review — анализ и исправление архитектурных проблем JavaScript и React

Блок 1. Производительность рендеринга и браузерный конвейер

Кейс 01. Принудительный сброс геометрии (Forced Reflow)

Проблема: Чтение и изменение высоты элементов происходит вперемешку. Это заставляет браузер многократно пересчитывать расположение элементов.

Исправленный код:

```
function alignCardHeights(cardsContainer) {
  const cards = cardsContainer.querySelectorAll(".product-card");

  // Сначала читаем все размеры
  const heights = Array.from(cards).map(card =>
    card.getBoundingClientRect().height
  );

  // Затем изменяем стили
  cards.forEach((card, index) => {
    if (heights[index] < 300) {
      card.style.height = "300px";
    }
  });
}
```

Объяснение: Браузер выполняет Style, Layout и Paint. Разделение чтения и записи позволяет браузеру группировать изменения и уменьшает количество перерасчётов страницы.

Кейс 02. Блокировка главного потока (Long Tasks)

Проблема: Обработка 100 000 транзакций занимает много времени и блокирует интерфейс.

Исправленный код:

```
async function processAnalytics(rawTransactions, filterCriteria) {
  const results = [];
  const chunkSize = 1000;

  for (let i = 0; i < rawTransactions.length; i += chunkSize) {
    const chunk = rawTransactions.slice(i, i + chunkSize);

    const processed = chunk
      .filter(t => t.amount > filterCriteria.min &&
                   t.status === "completed")
      .map(t => ({
        ...t,
        normalizedDate: new Date(t.timestamp).toLocaleDateString(),
        tax: t.amount * 0.2
      }));

    results.push(...processed);

    await new Promise(resolve => setTimeout(resolve, 0));
  }

  return results.reduce((acc, t) => {
    acc.totalRevenue += t.amount;
    acc.totalTax += t.tax;
    return acc;
  }, { totalRevenue: 0, totalTax: 0 });
}
```

Объяснение: Данные обрабатываются частями. Между частями браузер получает возможность обрабатывать пользовательские действия. Для ещё более тяжёлых вычислений можно использовать Web Worker.

Кейс 03. Неконтролируемая конкурентность запросов

Проблема: `Promise.all` запускает все 150 загрузок одновременно, создавая чрезмерную нагрузку.

Исправленный код:

```
async function asyncPool(tasks, limit = 4) {
  const results = new Array(tasks.length);
  let index = 0;

  async function worker() {
    while (index < tasks.length) {
      const current = index++;

      try {
        results[current] = {
          status: "fulfilled",
          value: await tasks[current]()
        };
      } catch (error) {
        results[current] = {
          status: "rejected",
          reason: error
        };
      }
    }
  }

  const workers = Array.from(
    { length: Math.min(limit, tasks.length) },
    () => worker()
  );

  await Promise.all(workers);
  return results;
}
```

Объяснение: Одновременно выполняется максимум 4 задачи. После завершения одной запускается следующая. Это снижает нагрузку на сервер.

Блок 2. Утечки памяти (Memory Leaks)

Кейс 04. Утечка памяти через глобальный кэш

Проблема: `Map` удерживает ссылки на DOM-элементы даже после их удаления со страницы.

Исправленный код:

```
const nodeMetadataCache = new WeakMap();

function attachComponentMetadata(domNode, metadata) {
  nodeMetadataCache.set(domNode, {
    ...metadata,
    attachedAt: Date.now()
  });
}
```

Объяснение: `WeakMap` хранит слабые ссылки на объекты. Если DOM-элемент больше нигде не используется, сборщик мусора может удалить его из памяти.

Кейс 05. Забытый AbortSignal

Проблема: При уходе со страницы запрос продолжает выполняться и удерживает ненужные данные.

Исправленный код:

```
function pollServerStatus(onStatusUpdate) {
  const controller = new AbortController();
  let isAlive = true;

  async function loop() {
    while (isAlive) {
      try {
        const res = await fetch("/api/health", {
          signal: controller.signal
        });

        const data = await res.json();
        onStatusUpdate(data);

      } catch (error) {
        if (error.name === "AbortError") break;
        console.error("Poll error", error);
      }

      if (isAlive) {
        await new Promise(resolve => setTimeout(resolve, 5000));
      }
    }
  }

  loop();

  return function unsubscribe() {
    isAlive = false;
    controller.abort();
  };
}
```

Объяснение: `AbortController` позволяет прервать выполняющийся запрос. При вызове `unsubscribe()` запрос отменяется.

Кейс 06. Утечка через замыкание

Проблема: Обработчик сохраняет ссылку на большой объект `hugeDataPayload`, хотя ему нужен только `iconId`.

Исправленный код:

```
function setupIconClickHandler(hugeDataPayload) {
  const iconId = hugeDataPayload.meta.iconId;

  hugeDataPayload = null;

  return function handleIconClick() {
    console.log("Clicked icon:", iconId);
  };
}
```

Объяснение: В обработчике используется только `iconId`. Ссылка на большой объект удаляется, поэтому он становится доступен для сборщика мусора, если других ссылок нет.

Кейс 07. Зомби-слушатель событий (Event Bus Zombie)

Проблема: При использовании `bind(this)` каждый раз создаётся новая функция. Поэтому отписка не работает.

Исправленный код:

```
class CartComponent {
  constructor(eventBus) {
    this.eventBus = eventBus;
    this.items = [];

    this.boundHandler = this.handleItemAdded.bind(this);

    this.eventBus.on("ITEM_ADDED", this.boundHandler);
  }

  handleItemAdded(item) {
    this.items.push(item);
  }

  destroy() {
    this.eventBus.off("ITEM_ADDED", this.boundHandler);
  }
}
```

Объяснение: Функция `bind()` создаёт новую ссылку при каждом вызове. Поэтому нужно сохранить привязанный обработчик и использовать его же при отписке.

Блок 3. Логические асинхронные аварии и состояние

Кейс 08. Устаревшее замыкание (Stale Closure)

Проблема: React использует старое значение `secondsLeft`, из-за чего таймер останавливается на 59.

Исправленный код:

```
useEffect(() => {
  const id = setInterval(() => {
    setSecondsLeft(prev => {
      if (prev <= 1) {
        clearInterval(id);
        return 0;
      }

      return prev - 1;
    });
  }, 1000);

  return () => clearInterval(id);
}, []);
```

Объяснение: Функциональное обновление `prev` получает актуальное значение состояния, а не старое значение из замыкания.

Кейс 09. Проглатывание ошибок (Silent Failure)

Проблема: При ошибке оплаты функция возвращает `undefined`, и пользователь не получает сообщение об ошибке.

Исправленный код:

```
async function processOrderPayment(paymentData) {
  try {
    const token = await generatePaymentToken(paymentData);
    const result = await chargeCard(token);

    return [null, result];

  } catch (error) {
    return [error, null];
  }
}

async function onCheckoutClick() {
  const [error, result] = await processOrderPayment(data);

  if (error) {
    showErrorModal(error.message);
    return;
  }

  showSuccessModal(result);
}
```

Объяснение: Функция возвращает ошибку и результат отдельно. Вызывающий код обязательно проверяет наличие ошибки.

Кейс 10. Гонка мутаций общего состояния

Проблема: Несколько компонентов одновременно запускают запросы и используют бесконечный цикл ожидания.

Исправленный код:

```
let cachedUser = null;
let userPromise = null;

async function getAuthenticatedUser() {
  if (cachedUser) return cachedUser;

  if (!userPromise) {
    userPromise = fetch("/api/me")
      .then(res => {
        if (!res.ok) throw new Error("Ошибка загрузки");
        return res.json();
      })
      .then(user => {
        cachedUser = user;
        return user;
      })
      .catch(error => {
        userPromise = null;
        throw error;
      });
  }

  return userPromise;
}
```

Объяснение: Вместо ожидания через `while` сохраняется один общий Promise. Все компоненты получают результат одного запроса. При ошибке кэш запроса сбрасывается.

Кейс 11. Бесконечные повторы запросов

Проблема: При ошибке функция повторяет запрос бесконечно, создавая высокую нагрузку на сервер.

Исправленный код:

```
async function fetchWithRetry(url, maxRetries = 5) {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error("HTTP error");
      }

      return await response.json();

    } catch (error) {
      if (attempt === maxRetries) throw error;

      const delay =
        Math.min(1000 * 2 ** attempt, 10000) *
        (0.5 + Math.random());

      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}
```

Объяснение: Используется Exponential Backoff — время ожидания увеличивается после каждой ошибки. Случайная задержка (Jitter) снижает вероятность одновременных повторов. Количество попыток ограничено.

Блок 4. Иммутабельность, нормализация и архитектура данных

Кейс 12. Неправильное глубокое клонирование объектов

Проблема: `JSON.parse(JSON.stringify())` теряет некоторые типы данных: функции, `undefined`, регулярные выражения и другие значения. Также не поддерживает `BigInt`.

Исправленный код:

```
const clonedState = structuredClone(state);
```

,

Объяснение: `structuredClone()` поддерживает глубокое копирование многих типов данных, включая `Date`, `RegExp` и `BigInt`. Однако функции копировать нельзя. Для обновления отдельных частей состояния лучше использовать Spread:

```
const newState = {
  ...state,
  balance: 2000
};
```

Кейс 13. Нарушение мемоизации React.memo

Проблема: При каждом рендере создаются новые объекты и функции. Поэтому `React.memo` считает свойства изменившимися.

Исправленный код:

```
const rowStyle = {
  padding: "8px",
  borderBottom: "1px solid #eee"
};

const rowOptions = {
  canSelect: true,
  canEdit: false
};

function Table({ rows }) {
  const handleDelete = React.useCallback((id) => {
    console.log("Delete", id);
  }, []);

  return (
    <div>
      {rows.map(row => (
        <TableRow
          key={row.id}
          data={row}
          style={rowStyle}
          options={rowOptions}
          onDelete={handleDelete}
        />
      ))}
    </div>
  );
}
```

Объяснение: Объекты создаются один раз вне компонента, а `useCallback` сохраняет ссылку на функцию между рендерами. Это позволяет `React.memo` эффективнее пропускать ненужные обновления.

Кейс 14. Дублирование элементов при пагинации

Проблема: При загрузке новых страниц могут повторяться записи с одинаковым `id`.

Исправленный код:

```
function appendNextPage(feedState, newPostsFromApi) {
  const existingIds = new Set(
    feedState.posts.map(post => post.id)
  );

  const uniquePosts = newPostsFromApi.filter(post => {
    if (existingIds.has(post.id)) return false;

    existingIds.add(post.id);
    return true;
  });

  return {
    ...feedState,
    posts: [...feedState.posts, ...uniquePosts]
  };
}
```

Объяснение: `Set` позволяет быстро проверять наличие идентификаторов. Дубликаты исключаются, а исходный массив не изменяется. Сложность алгоритма — O(N).

Кейс 15. Неправильная обработка NULL при деструктуризации

Проблема: Значения по умолчанию в деструктуризации работают только при `undefined`, но не при `null`.

Исправленный код:

```
function renderBadgeConfig(config = {}) {
  const color = config.color ?? "blue";
  const text = config.text ?? "Новинка";
  const icon = config.icon ?? "star";

  return `<span class="badge ${color}">${icon} ${text}</span>`;
}
```

Объяснение: Оператор `??` подставляет значение по умолчанию, если исходное значение равно `null` или `undefined`.

Итог практической работы

В ходе выполнения рассмотрены 15 архитектурных проблем JavaScript и React:

1. Исправление принудительного пересчёта геометрии.
2. Оптимизация тяжёлых вычислений.
3. Ограничение параллельных запросов.
4. Предотвращение утечек памяти через WeakMap.
5. Отмена сетевых запросов через AbortController.
6. Исправление утечек через замыкания.
7. Правильная отписка от событий.
8. Исправление устаревших замыканий.
9. Обработка асинхронных ошибок.
10. Дедупликация сетевых запросов.
11. Ограничение повторных запросов.
12. Безопасное клонирование объектов.
13. Оптимизация React.memo.
14. Удаление дубликатов в списках.
15. Нормализация значений NULL.

Вывод: В результате работы изучены способы повышения производительности приложений, предотвращения утечек памяти и улучшения архитектуры JavaScript и React.